

CENG218 – Labwork 11 The Priority Queue Abstract Data Type

A priority queue is an abstract data type for storing a collection of prioritized elements that supports arbitrary element insertion, but supports removal of elements in order of priority, that is, the element with first priority can be removed at any time.

The Priority Queue ADT

As an ADT, a priority queue P supports the following functions:

- `isPQueueEmpty()`: returns true if pqueue is empty otherwise false
- `size()`: Return the number of elements in P .
- `empty()`: Return true if P is empty and false otherwise.
- `insert(e)`: Insert a new element e into P .
- `min()`: Return a reference to an element of P with the smallest associated key value (but do not remove it); an error condition occurs if the priority queue is empty.
- `removeMin()`: Remove from P the element referenced by `min()`; an error condition occurs if the priority queue is empty.

Example : The following table shows a series of operations and their effects on an initially empty priority queue P . Each element consists of an integer, which we assume to be sorted according to the natural ordering of the integers. Note that each call to `min` returns a reference to an entry in the queue, not the actual value. Although the “Priority Queue” column shows the items in sorted order, the priority queue need not store elements in this order.

Operation	Return	Priority Queue
<code>insert(5)</code>	-	{5}
<code>insert(9)</code>	-	{5,9}
<code>insert(2)</code>	-	{2,5,9}
<code>insert(7)</code>	-	{2,5,7,9}
<code>min()</code>	[2]	{2,5,7,9}
<code>removeMin()</code>	[2]	{5,7,9}
<code>size()</code>	3	{5,7,9}
<code>min()</code>	[5]	{5,7,9}
<code>removeMin()</code>	[5]	{7,9}
<code>removeMin()</code>	[7]	{9}
<code>removeMin()</code>	[9]	{}
<code>empty()</code>	true	{}
<code>removeMin()</code>	invalid operation	{}

Implementing a Priority Queue with a List

In this section, we show how to implement a priority queue by storing its elements in an Linked list. We may consider two realizations, depending on whether we sort the elements of the list.

1. Implementation with an Unsorted List

Let us first consider the implementation of a priority queue P by an unsorted linked list L . A simple way to perform the operation `insert(e)` on P is by adding each new element at the beginning of L . This implementation of `insert` takes $O(1)$ time.

Since the insertion does not consider key values, the resulting list L is unsorted. As a consequence, in order to perform either of the operations `min` or `removeMin` on P , we must inspect all the entries of the list to find one with the minimum key value. Thus, functions `min` and `removeMin` take $O(n)$ time each, where n is the number of elements in P at the time the function is executed.

2. Implementation with a Sorted List

An alternative implementation of a priority queue P also uses a list L , except that this time let us store the elements sorted by their key values. Specifically, we represent the priority queue P by using a list L of elements sorted by nondecreasing key values, which means that the first element of L has the smallest key.

We can implement function `min` in this case by accessing the element associated with the first element of the list L . Likewise, we can implement the `removeMin` function of P by removing first node of list L . Assuming that L is implemented as a linked list, operations `min` and `removeMin` in P take $O(1)$ time, so are quite efficient.

Following table compares the running times of the functions of a priority queue realized by means of an unsorted and sorted list, respectively. There is an interesting contrast between the two functions. An unsorted list allows for fast insertions but slow queries and deletions, while a sorted list allows for fast queries and deletions, but slow insertions.

Operation	Unsorted List	Sorted List
size, empty	$O(1)$	$O(1)$
insert	$O(1)$	$O(n)$
min, removeMin	$O(n)$	$O(1)$

Labwork 9 Task 1

Implement Priority Queue using Linked List and perform 100,000 random value insertions while calculating running time (number of iterations) for each operation and record them.

Labwork 9 Task 2

Implement Priority Queue using MinHeap, and perform 100,000 random value insertions while calculating running time (number of iterations) for each operation and record them.

Run Time of a Program

The program performance is the amount of time needed to run a program and can be estimated analytically or observed experimentally. In order to observe running time of our PQueue implementation, we shall design an experiment such that we will be able to make conclusion about running time performance of our implementation (sorted and unsorted linked list).

Why do want to do a performance analysis of algorithms?

- To determine the practicality of algorithm
- To predict run time on large instance
- To compare two algorithms that have different implementations

The running time of our algorithm can be effected by cpu speed and number of inputs. Since cpu performance will be fixed during experiment only number of inputs (input size) effects running time of our algorithm. Now Please make necessary modifications to measure running times given in following table.

Input Size(n)	Time(sorted)
10,000 (enqueue operations)	
20,000	
30,000	
40,000	
50,000	
60,000	
70,000	
80,000	
Input Size(n)	Time
10,000 (deleteMin operations)	
20,000	
30,000	
40,000	
50,000	
60,000	
70,000	
80,000	

How to measure wall clock of a program

```
double get_CPU_time_usage(clock_t clock1, clock_t clock2)
{
    double diffticks=clock1-clock2;
    double diffms=(diffticks*1000)/(CLOCKS_PER_SEC*1000);
    return diffms;
}
```

```
main(){
// clock ticks per millisecond
clock_t startTime, endTime;
startTime = clock();
// code to be timed comes here
//
endTime= clock();

// elapsed time in milliseconds
double elapsedTime=get_CPU_time(endTime,startTime);
```